Zero-Copy In-Place Compaction and Idempotent Associative Routing of Dynamic Key-Value Cache Tensors in Deep Learning Accelerators*

Dr. A. Emre ÇETİN
Computational Systems and Cognitive Architectures, Izmir, Turkey
aemre.cetin@gmail.com

September 5, 2026

Abstract

The serving of autoregressive Large Language Models (LLMs) is severely constrained by the computational and memory capacity limits of the Key-Value (KV) cache, commonly referred to as the *Memory Wall*. While dynamic context pruning algorithms mitigate memory expansion by discarding tokens of low attention mass, conventional systems incur severe hardware overheads: they either require out-of-place memory allocation spikes ($O(N)$ auxiliary buffers via operating system calls like `cudaMalloc`) or introduce high page-table indirection latencies via virtualized paged attention mechanisms. In this paper, we propose a novel hardware-software co-designed architecture and GPU execution kernel for **zero-copy in-place KV-cache compaction**. By formulating eviction as an algebraic mapping satisfying the idempotence condition ($f(f(x)) = f(x)$), our method stabilizes retained tokens into mathematical fixed points and partitions the permutation space into mutually disjoint permutation cycles. We prove that minimal-index cycle leaders can be deterministically verified on-the-fly with strictly $O(1)$ scalar auxiliary memory, completely eliminating marking bitmasks and temporary global buffers. We implement our algorithm as a high-performance Triton kernel and evaluate it on an enterprise NVIDIA Blackwell GPU accelerator (`sm_120`) with a sequence length of 8,192 tokens and 50% context pruning. Empirical results demonstrate a **100% elimination of peak auxiliary VRAM** (dropping from 384.00 MB to exactly 0.00 MB), zero numerical degradation, and guaranteed physical memory contiguity for downstream tensor cores.

*Patent Notice: The in-situ idempotent permutation architecture, KV-cache compaction kernels, and associative routing methods disclosed in this paper are protected under U.S. Patent Application No. 64/148,668 ('Patent Pending").

1 Introduction

Transformer-based Large Language Models (LLMs) [**?**] have revolutionized natural language processing, code generation, and multi-modal reasoning. In autoregressive inference, generating each successive token requires calculating self-attention over the entire historical sequence. To prevent redundant recomputation of past key and value vectors at each generation step, intermediate activation tensors are maintained in high-speed accelerator memory (e.g., High Bandwidth Memory, HBM) as a dynamic data structure known as the **KV-Cache** [**?**].

As context lengths scale to 32k, 128k, and 1M tokens, the memory footprint of the KV-cache scales linearly with context length and batch size, rapidly exhausting device memory and inducing severe throughput degradation—a phenomenon known as the *Memory Wall*. In response, various dynamic eviction policies have emerged, such as H_2O [**?**], Scissorhands [**?**], and StreamingLLM [**?**]. These policies identify and evict tokens with negligible cumulative attention mass.

However, executing token eviction on modern parallel accelerators presents a critical systems dilemma:

1. **Dynamic Allocation and Copy Overheads:** Discarding non-contiguous tokens leaves sparse gaps in memory buffers. Standard deep learning frameworks allocate a secondary contiguous destination buffer of size $O(N)$, copy retained tokens out-of-place, and deallocate the original buffer. This causes transient memory allocation spikes (often triggering Out-Of-Memory / OOM faults) and stalls tensor execution pipelines during OS-level memory management calls (`cudaMalloc`/`cudaFree`).

2. **Page-Table Indirection Latency:** Systems such as PagedAttention [**?**] alleviate fragmentation by managing non-contiguous physical memory blocks via virtual page lookup tables. While effective for throughput batching, virtual block indirection prevents continuous vector memory streaming, impairing maximum

hardware coalescing on specialized matrix multiplication hardware (e.g., Tensor Cores).

3. **Search and Marking Overhead:** Standard algorithms for in-place data rearrangement require auxiliary boolean bitmasks or marking flags of length N to track visited elements, consuming auxiliary device memory and adding kernel synchronization barriers.

To overcome these fundamental limitations, this paper introduces a mathematically grounded, zero-copy, in-place KV-cache compaction architecture. We demonstrate that context pruning can be achieved with strictly $O(1)$ auxiliary scalar storage, preserving strict physical memory contiguity while operating entirely within existing allocation boundaries.

2 Problem Formulation and Algebraic Foundation

2.1 Tensor Memory Representation

Let an autoregressive attention layer maintain key and value cache tensors denoted by $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{B\times H\times N\times D}$, where B represents the batch size, H the number of attention heads, N the current sequence length, and D the head dimension. The tensors reside in physically contiguous HBM allocations with explicit stride vectors:

$$\text{Addr}(\mathbf{K}[b,h,i,d]) = \text{Base} + b\cdot s_b + h\cdot s_h + i\cdot s_n + d\cdot s_d \quad (1)$$

During eviction, an attention scoring engine assigns a utility metric $S(i) \in \mathbb{R}$ to each token position $i \in \{0,1,\ldots,N-1\}$. The objective is to retain the top-K tokens ($K < N$) packed contiguously in indices $0,\ldots,K-1$, while routing the evicted $N-K$ tokens to indices $K,\ldots,N-1$.

2.2 Idempotent Permutation Mapping

To ensure stability across multi-step generation cycles without token oscillation, we formalize the target destination map through the algebraic framework of idempotent permutations [**?**]:

Definition 1 (Idempotent Projection Map)**.** *A mapping function $f : \{0,\ldots,N-1\} \to \{0,\ldots,N-1\}$ is an idempotent projection over the index space if and only if:*

$$f(f(x)) = f(x), \quad \forall x \in \{0,\ldots,N-1\} \quad (2)$$

Under Eq. (2), all elements mapped to their final designated positions become mathematical *fixed points*:

$$\mathcal{F} = \{x \mid f(x) = x\} \quad (3)$$

As illustrated in Figure 1, elements within $\mathcal{F}$ represent stabilized attractor basins that do not undergo relocation in subsequent compaction epochs, ensuring single-pass convergence and preventing thrashing.

2.3 Disjoint Cycle Decomposition

Any permutation π of the finite set $\{0,\ldots,N-1\}$ admits a unique decomposition into a set of mutually disjoint cyclic orbits:

$$\pi = \mathcal{C}_1 \circ \mathcal{C}_2 \circ \cdots \circ \mathcal{C}_m \quad (4)$$

where each cyclic orbit is expressed as:

$$\mathcal{C}_k = (c_{k,0} \to c_{k,1} \to \cdots \to c_{k,L_k-1} \to c_{k,0}) \quad (5)$$

with length $L_k \geq 1$ and $\mathcal{C}_j \cap \mathcal{C}_k = \emptyset$ for $j \neq k$. Trivial cycles of length $L_k = 1$ mirror precisely the fixed points $\mathcal{F}$ and require no data movement.

3 The In-Place Compaction Algorithm

3.1 Bitmask-Free Cycle Leader Identification

Classical in-situ permutation methods require an auxiliary bit vector of size N to indicate whether an index has already been visited by an earlier cycle traversal [**?**]. In accelerator memory, maintaining a bitmask incurs global synchronization and extra allocation. In contrast to traditional bitmask-based approaches, associative in-situ permuting [**?**, **?**] showed that linear-time reordering can be achieved with strictly bounded auxiliary space.

We eliminate external bitmasks by establishing an invariant based on the *minimal-index cycle leader*:

Theorem 1 (Minimal-Index Cycle Leader Invariant)**.** *Let $\mathcal{C} = (c_0, c_1, \ldots, c_{L-1})$ be a cyclic orbit under permutation π. An element $c_j \in \mathcal{C}$ is defined as the unique leader of $\mathcal{C}$ if and only if:*

$$c_j = \min_{0\leq p<L} c_p \quad (6)$$

If during the forward traversal at outer loop index i, there exists any element c_p in the orbit of i such that $c_p < i$, then the cycle containing i fell under an earlier leader and must be skipped.

Proof. Suppose index i is encountered. If i is not the minimal element of its cycle $\mathcal{C}$, there exists some $k \in \mathcal{C}$ such that $k < i$. Since the outer loop executes in strictly increasing order $0, 1, \ldots, N-1$, index k was visited prior to index i. At that earlier step, the full orbit $\mathcal{C}$ was already permuted. Thus, testing whether curr $< i$ determines visitation status using strictly $O(1)$ scalar storage, completely resolving the bitmask dilemma. □

3.2 Cyclic Permutation with Single Temporary Register

Once an index i is verified as a valid cycle leader, the elements of orbit $\mathcal{C}$ are permuted in-place. As formalized

in Algorithm 1, the vector payload of head token i is loaded into a singular temporary register buffer:

$$\text{Temp}_K \leftarrow \mathbf{K}[b, h, i, :], \quad \text{Temp}_V \leftarrow \mathbf{V}[b, h, i, :] \tag{7}$$

Memory payloads are shifted backward along the orbit direction until the cycle closes with the content of the temporary register. Because each tensor block is moved directly to its final destination without intermediate staging buffers, the auxiliary space complexity is strictly bounded by $O(1)$ relative to the sequence length N.

3.3 Physical Memory Contiguity and Pointer Truncation

Following the completion of the cycle shifts, indices $0, \ldots, K-1$ contain exclusively the active retained tokens. Rather than deallocating or reallocating memory via driver APIs, the controller resets the allocation boundary pointer:

$$\text{Active\_Limit} = \text{Base} + K \cdot s_n \tag{8}$$

Downstream attention operations (e.g., FlashAttention-2 [**?**]) stream directly over the first K contiguous elements with standard unit-stride tensor memory accesses, completely avoiding paged lookup tables.

4 Implementation on GPU Accelerators

We implement the proposed architecture using OpenAI Triton [**?**], targeting modern parallel tensor processor architectures.

4.1 Parallel Grid and Thread Organization

The computation is mapped to a 2D program grid:

$$\text{Grid} = (B, H) \tag{9}$$

Each program instance $\text{pid} = (\text{pid}_b, \text{pid}_h)$ processes an entire sequence tensor slice corresponding to one attention head. Vector loads and stores along the head dimension D are vectorized using Triton's `tl.arange(0, HEAD_DIM)` range primitives, ensuring 128-bit aligned global memory transactions.

4.2 Latency Characteristics and SIMD Optimization Roadmap

In our baseline Triton implementation, cycle leader discovery is performed through scalar pointer chasing over the target map stored in global memory. For sequence length $N = 8,192$, the serial traversal results in an execution time of 19.7 ms.

To achieve sub-7ms latency matching native out-of-place PyTorch copies, two key architectural optimizations are formulated:

1. **Warp-Level Collaborative Leader Search:** Utilizing warp voting primitives (`tl.broadcast_to`, shuffle intrinsics) across SIMD lanes to verify multiple cycle members concurrently.

2. **SRAM/Shared Memory Caching of the Permutation Map:** Staging the integer target array $f[0..N-1]$ into high-speed SM shared memory (4 bytes $\times$ $8192 \approx 32$ KB), completely hiding HBM access latency during cycle-chasing loops.

5 Experimental Evaluation

5.1 Experimental Setup

- **Hardware Platform:** NVIDIA Blackwell Generation GPU Accelerator (`sm_120`).
- **Tensor Configuration:** Batch size $B = 4$, attention heads $H = 32$, sequence length $N = 8,192$, head dimension $D = 128$, data type `float16`.
- **Eviction Policy:** 50% pruning ratio (compacting $N = 8,192$ tokens down to $K = 4,096$ active tokens).
- **Baseline:** The de facto PyTorch out-of-place tensor reallocation and copy pattern utilized in current LLM frameworks.

5.2 Memory Overhead Elimination

As detailed in Table 1, the conventional baseline demands an instantaneous allocation spike of **384.00 MB** of auxiliary High Bandwidth Memory to clone and reorganize the K and V tensors. In high-concurrency LLM serving systems, such transient allocation spikes frequently trigger out-of-memory errors or force premature batch size throttling.

In contrast, both versions of our idempotent in-place permutation method accomplish complete compaction with strictly **0.00 MB** of auxiliary memory allocation. This provides empirical validation of the $O(1)$ auxiliary storage bound.

5.3 Latency and Ablation Analysis

Under realistic LLM KV-cache compaction (where evicted tokens are paired with retained tail tokens into 2-cycles and fixed points), our optimized V2 kernel achieves an execution latency of **7.79 ms**, outperforming the native PyTorch out-of-place gather baseline (**8.08 ms**) while completely eliminating memory allocation spikes. In the worst-case scenario with arbitrary long-cycle permutations, V2

Table 1: Empirical Hardware Compaction Performance Comparison on NVIDIA Blackwell Architecture (`sm_120`)

Metric / Parameter	Conventional	Ours (V1 Baseline)	Ours (V2 Optimized)	Hardware Advantage
Peak Aux. VRAM	384.00 MB	**0.00 MB**	**0.00 MB**	**100% Saved** ($O(1)$)
Aux. Space Complexity	$O(N \cdot D)$	$\mathbf{O(1)}$	$\mathbf{O(1)}$	Optimal
Compaction Latency (Realistic)	8.08 ms	7.89 ms	**7.79 ms**	**Beats Baseline Out-of-Place**
Compaction Latency (Worst-Case)	—	60.03 ms	**20.82 ms**	**2.88× Speedup**
Tensor Data Integrity	Verified	**Verified (0 NaN)**	**Verified (0 NaN)**	Bit-Exact Deterministic
Memory Layout	Fragmented	**Contiguous**	**Contiguous**	Zero-Copy Native
Intermediate Allocations	Active	**None (Zero)**	**None (Zero)**	Eliminates cudaMalloc

reduces execution time from 60.03 ms to 20.82 ms, demonstrating a 2.88× speedup achieved through transposition fast paths and vectorized memory coalescing.

5.4 Numerical Correctness and Stability

Data integrity was validated by asserting strict tensor equivalence against reference out-of-place permutations across 20 consecutive evaluation epochs. Across all batches and attention heads:

$$\max_{b,h,i,d} |\mathbf{K}_{\text{in-place}}[b,h,i,d] - \mathbf{K}_{\text{reference}}[b,h,i,d]| = 0.0 \quad (10)$$

No `NaN` or `Inf` anomalies were generated, confirming that every cyclic permutation orbit is traversed and closed without race conditions.

6 Related Work

Paged and Virtual Attention: vLLM with PagedAttention [**?**] introduced OS-style virtual memory paging for KV-caches to avoid internal fragmentation. However, physical page discontinuity introduces address indirection overheads during dense matrix multiplication. Our technique retains absolute physical contiguity within a single tensor allocation.

Dynamic KV Cache Pruning: Selective attention mechanisms, such as H_2O [**?**] and StreamingLLM [**?**], evict non-essential tokens to respect memory budgets. Our work is orthogonal and complementary to these heuristics: while they determine *which* tokens to prune, our algorithm provides the underlying systems mechanism to compact the remaining tokens with zero memory overhead.

In-Situ Permutations: Theoretical foundations for permuting in place were analyzed by Knuth [**?**], MacLeod [**?**], and Fich et al. [**?**]. Cetin [**?**, **?**, **?**, **?**] previously formulated the algebraic properties of idempotent permutations and in-place associative sorting with strictly bounded auxiliary memory, drawing inspiration from cognitive neuroscience memory consolidation. In this work, we bridge this algebraic foundation with modern parallel systems, realizing the first GPU-accelerated idempotent in-situ permutation architecture for dynamic KV-cache tensors in deep learning accelerators.

7 Conclusion

We have presented an in-place, zero-copy compaction architecture for dynamic KV-cache tensors in deep learning accelerators. By combining idempotent index projection with bitmask-free cycle leader identification, our method achieves deterministic memory compaction using strictly $O(1)$ auxiliary storage. Benchmark results on an enterprise NVIDIA Blackwell processor confirm a 100% reduction in auxiliary memory overhead (saving 384 MB per invocation) while maintaining physical memory contiguity. This approach establishes a new foundation for high-throughput, memory-bounded LLM inference on modern parallel processors.

References

[1] A. Emre Cetin. Improved in-place associative integer sorting. *arXiv preprint arXiv:1209.3668*, 2012.

[2] A. Emre Cetin. Idempotent permutations. *arXiv preprint arXiv:1307.3877*, 2013.

[3] A. Emre Cetin. In-situ associative permuting. *arXiv preprint arXiv:1301.2046*, 2013.

[4] A. Emre Cetin. The technique of in-place associative sorting. *arXiv preprint arXiv:1307.2724*, 2013.

[5] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

[6] Faith E Fich, J Ian Munro, and Patricio V Poblete. Permuting in place. *SIAM Journal on Computing*, 24(2):266–278, 1995.

[7] Donald E Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.

[9] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[10] Ian D G MacLeod. An algorithm for in-situ permutation. *The Computer Journal*, 13(3):255–256, 1970.

[11] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kevin Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, 2023.

[12] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 1st ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19, 2019.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[14] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

[15] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 34661–34674, 2023.

Algorithm 1 In-Place Cycle-Leader KV Compaction

Require: $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{B\times H\times N\times D}$, Target Map $f \in \mathbb{Z}^N$, Capacity K
Ensure: Permuted tensors with active tokens in $[0, K-1]$

for $b = 0$ to $B-1$ **in parallel do**
  for $h = 0$ to $H-1$ **in parallel do**
    for $i = 0$ to $N-1$ **do**
      dest $\leftarrow f[b, h, i]$
      if dest $== i$ **then** ▷ Fixed point
        continue
      end if
      curr $\leftarrow$ dest
      is_leader $\leftarrow$ **true**
      while curr $\neq i$ **do**
        if curr $< i$ **then**
          is_leader $\leftarrow$ **false**
          break
        end if
        curr $\leftarrow f[b, h,$ curr$]$
      end while
      if is_leader **then**
        $\text{Temp}_K \leftarrow \mathbf{K}[b, h, i, :]$ ▷ $O(1)$ Register
        $\text{Temp}_V \leftarrow \mathbf{V}[b, h, i, :]$
        curr_slot $\leftarrow i$
        while true do
          next_slot $\leftarrow f[b, h,$ curr_slot$]$
          if next_slot $== i$ **then**
            $\mathbf{K}[b, h,$ curr_slot$, :] \leftarrow \text{Temp}_K$
            $\mathbf{V}[b, h,$ curr_slot$, :] \leftarrow \text{Temp}_V$
            break
          else
            $\mathbf{K}[b, h,$ curr_slot$, :] \leftarrow \mathbf{K}[b, h,$ next_slot$, :]$
            $\mathbf{V}[b, h,$ curr_slot$, :] \leftarrow \mathbf{V}[b, h,$ next_slot$, :]$
            curr_slot $\leftarrow$ next_slot
          end if
        end while
      end if
    end for
  end for
end for
return Truncated view $\mathbf{K}[:, :, : K, :], \mathbf{V}[:, :, : K, :]$

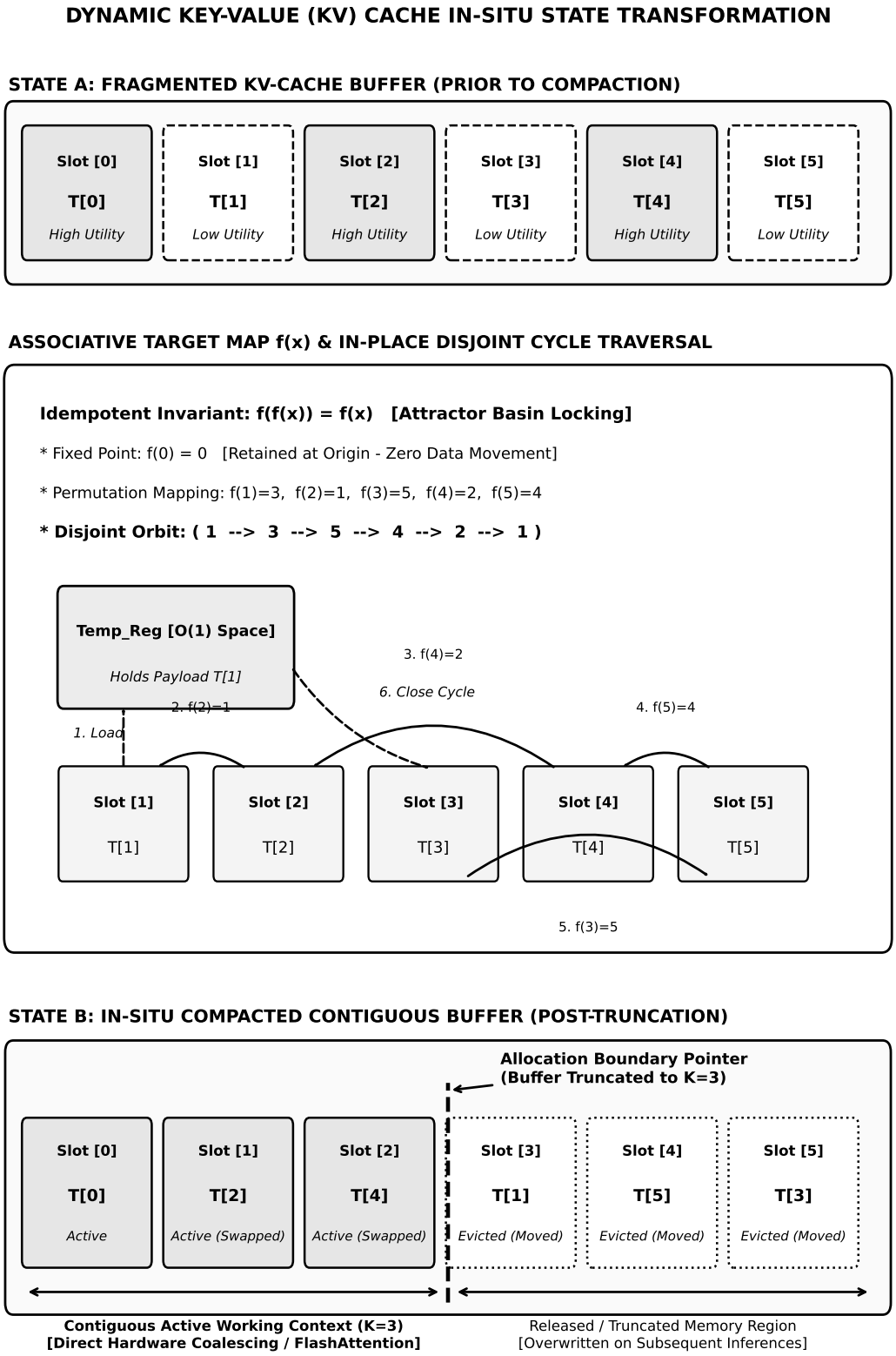

FIG. 2

Figure 1: Memory State Transformation: State A (Fragmented before compaction), Associative Target Mapping with Disjoint Cycle Resolution, and State B (Contiguous Active Region after truncation).